Parallel Matrix Multiplication using MPI

• Introduction

Matrix multiplication is one of the most important operations in computer science, mathematics, data analysis, and scientific computing. However, when the matrix size becomes large, the calculation can take a long time because each element of the result matrix requires multiple multiplications and additions. For this reason, matrix multiplication is a good example to study the benefit of parallel computing.

In this project, I implemented matrix multiplication using MPI. First, I used a sequential approach, where one processor performs all the calculations. Then, I implemented a parallel approach, where the rows of matrix A are divided among multiple processors. Each processor calculates part of the result matrix, and processor 0 collects the final results. The purpose of this project is to compare execution time using different numbers of processors and observe how parallelism improves performance.

• Sequential Algorithm

In the sequential approach, matrix multiplication is performed using a single processor. The processor computes every element of the resulting matrix one by one without any parallelism. For two matrices A and B of size N × N, each element of the result matrix C is calculated using the dot product of a row from matrix A and a column from matrix B.

The computation follows three nested loops. The outer loop iterates through the rows of matrix A, the middle loop iterates through the columns of matrix B, and the inner loop performs the multiplication and summation of corresponding elements. As a result, the total time complexity of the sequential algorithm is $O(N^3)$, which becomes very expensive for large matrix sizes.

In this project, the sequential version is represented by running the program with only one processor (np = 1). In this case, no communication between processors is required, and all computations are handled by processor 0. This approach is simple but inefficient for large-scale problems because it does not utilize multiple processors.

• Parallel Algorithm

In the parallel approach, the matrix multiplication is divided among multiple processors using the Message Passing Interface (MPI). Instead of one processor performing all computations, the workload is distributed so that each processor calculates a portion of the result matrix.

The parallel algorithm follows a row-wise decomposition strategy. Matrix A is divided into subsets of rows, and each processor is assigned a group of rows to compute. Processor 0 is responsible for initializing matrices A and B and distributing the data to other processors using MPI communication functions.

First, processor 0 sends the entire matrix B to all other processors using MPI_Send, since every processor needs matrix B to perform multiplication. Then, it distributes different rows of matrix A to each processor. After receiving the data, each processor independently computes its assigned rows of the result matrix C using the same multiplication logic as in the sequential algorithm.

Once the computation is complete, each processor sends its partial results back to processor 0 using `MPI_Send`. Processor 0 collects all computed rows using `MPI_Recv` and combines them to form the final result matrix.

This parallel approach significantly reduces execution time because multiple processors work simultaneously. However, the performance improvement is not perfectly linear due to communication overhead and synchronization between processors.

• Experimental Results

To evaluate the performance of the parallel matrix multiplication algorithm, experiments were conducted using different numbers of processors. The matrix size was set to N = 500 to ensure that the computation time is significant and measurable.

The execution time was measured using the MPI_Wtime() function, which provides high-resolution timing. The program was executed with 1, 2, 4, 5, and 10 processors. Each configuration was tested multiple times, and the average execution time was recorded to improve accuracy.

Processors	Time (seconds)
1	1.73
2	1.57
4	1.54
5	1.45
10	1.39

Speedup Curve:

Speedup = T(1) / T(p)

P	Time	Speedup
1	1.73	1.00
2	1.57	1.10
4	1.54	1.12
5	1.45	1.19
10	1.39	1.24

Efficiency Curve

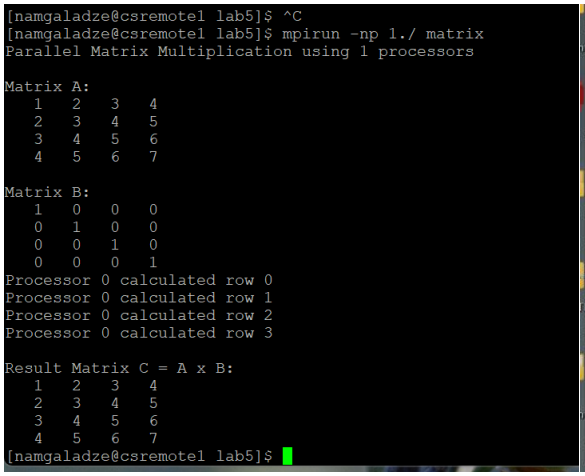
Efficiency = Speedup / P

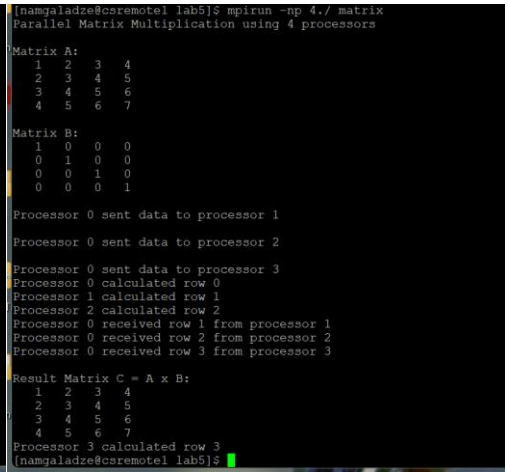
P	Efficiency
1	1.00
2	0.55
4	0.28
5	0.24
10	0.12

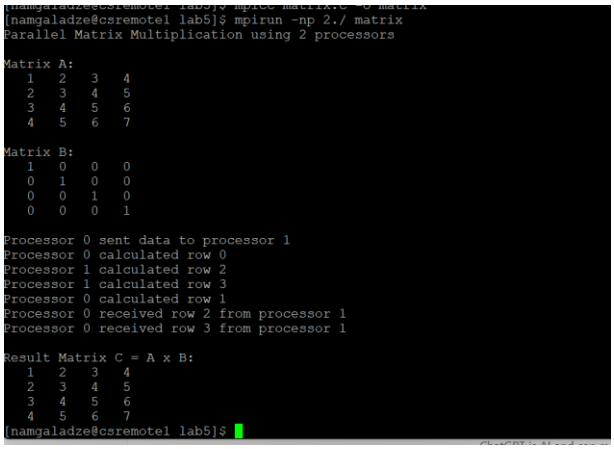
In addition to execution time, performance metrics such as speedup and efficiency were analyzed. Speedup measures how much faster the parallel algorithm is compared to the sequential version, while efficiency shows how effectively the processors are utilized.

The results indicate that speedup increases as the number of processors increases. However, efficiency decreases due to communication overhead and synchronization between processors. This demonstrates that although parallelization improves performance, the benefit is limited by system overhead, Ideally, speedup should increase linearly, but in practice it is limited due to communication overhead.

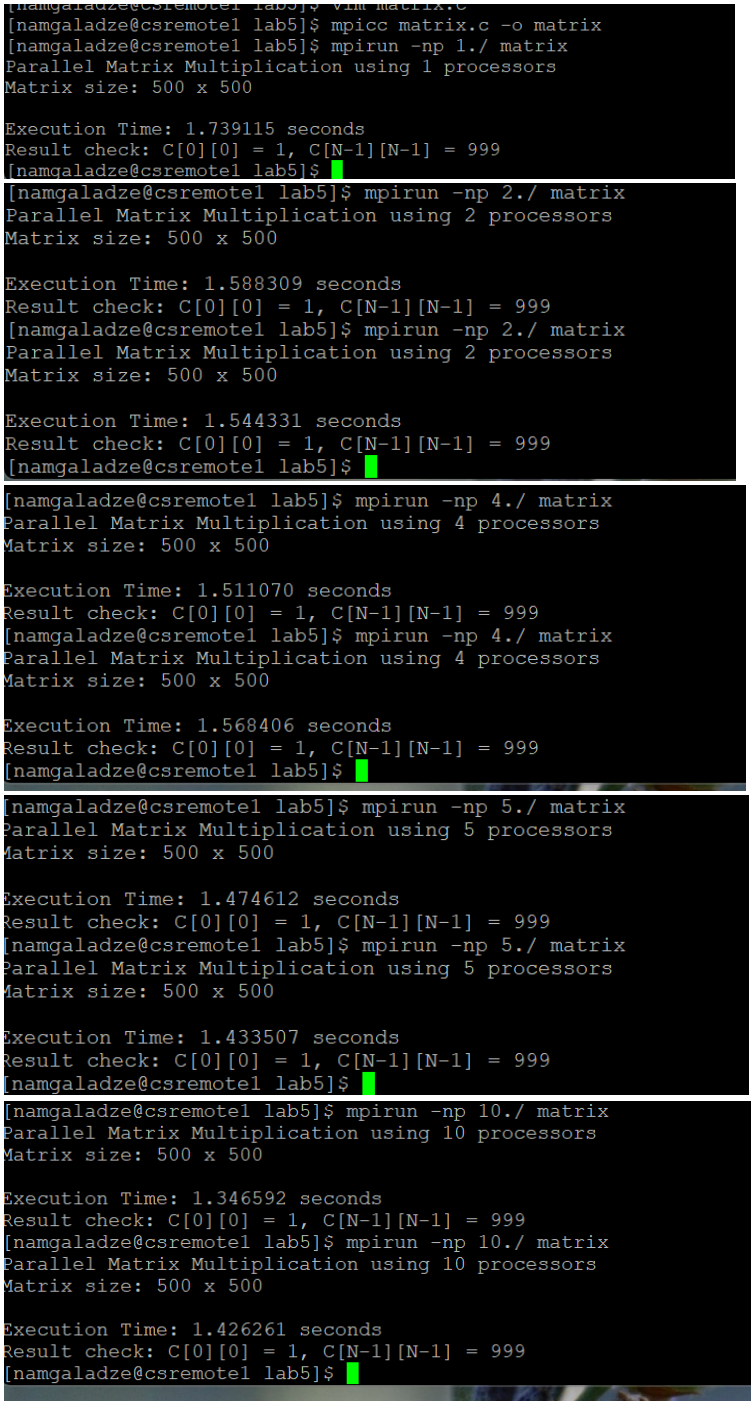
N = 4 (matrix output)







N = 500 (execution time)



The graph shows the relationship between the number of processors and execution time. As the number of processors increases, the execution time decreases. This demonstrates that parallel processing improves performance by distributing the workload among multiple processors. However, the improvement is not perfectly linear due to communication overhead and synchronization between processes.

- # Conclusion

In this project, matrix multiplication was implemented using both sequential and parallel approaches with MPI. The results demonstrate that parallel computation significantly improves performance by distributing the workload among multiple processors.

As the number of processors increases, the execution time decreases, confirming the effectiveness of parallel processing. However, the improvement is not perfectly linear due to communication overhead and synchronization between processors. These factors introduce additional costs that limit the overall speedup.

Overall, this project highlights the advantages of parallel computing for computationally intensive tasks and demonstrates how MPI can be used to efficiently divide and manage workloads across multiple processors.